CS3423 Compilers II
Mini-Assignment 4: Clang AST Passes

CS24BTECH11030, CS24BTECH11036

October 2026

Contributions. CS24BTECH11030: Part 1 (AST dump and node analysis) and the traversal mechanisms section. CS24BTECH11036: Part 2 (the three AST-matcher passes, their tests and write-up).

1 Part 1: Exploring AST dumps

1.1 Program

The input is `nested_loops.cpp`: a function `f` containing a loop, and a `main` with a doubly nested loop (both loops use a variable called `i`) and a call to `f`.

```
#include <stdio.h>

void f(int n){
    for(int i = 0; i < n; i++){
        printf("Hello World\n");
    }
}

int main(){
    int n = 10;
    for(int i = 0; i < n; i++){
        for(int i = 0; i < n; i++){
            printf("Hello World\n");
        }
    }
    f(n);
    return 0;
}
```

1.2 Dumping the AST

```
clang -fsyntax-only -Xclang -ast-dump -Xclang -ast-dump-filter=main nested_loops.cpp
```

`-ast-dump-filter=main` restricts the dump to the declaration named `main`; without it the headers pulled in by `<stdio.h>` dominate the output. The dump below has addresses and most source ranges removed for readability (the raw output is `ast_dump_main.txt`).

```
FunctionDecl main 'int ()'
`-CompoundStmt
  |-DeclStmt
```

```
| `-VarDecl used n 'int' cinit
|   `-IntegerLiteral 'int' 10
|-ForStmt                                  <-- outer loop
| |-DeclStmt
| | `-VarDecl used i 'int' cinit
| |   `-IntegerLiteral 'int' 0
| |-<<<NULL>>>                             <-- no condition variable
| |-BinaryOperator 'bool' '<'
| | |-ImplicitCastExpr 'int' <LValueToRValue>
| | | `-DeclRefExpr 'int' lvalue Var 'i'
| | `-ImplicitCastExpr 'int' <LValueToRValue>
| |   `-DeclRefExpr 'int' lvalue Var 'n'
| |-UnaryOperator 'int' postfix '++'
| | `-DeclRefExpr 'int' lvalue Var 'i'
| `-CompoundStmt
|   `-ForStmt                              <-- inner loop
|     |-DeclStmt ... (VarDecl i)
|     |-<<<NULL>>>
|     |-BinaryOperator 'bool' '<'
|     |-UnaryOperator 'int' postfix '++'
|     `-CompoundStmt
|       `-CallExpr 'int'                    <-- printf(...)
|         |-ImplicitCastExpr <FunctionToPointerDecay>
|         | `-DeclRefExpr Function 'printf'
|         `-ImplicitCastExpr <ArrayToPointerDecay>
|           `-StringLiteral "Hello World\n"
|-CallExpr 'void'                           <-- f(n)
| |-ImplicitCastExpr <FunctionToPointerDecay>
| | `-DeclRefExpr Function 'f'
| `-ImplicitCastExpr <LValueToRValue>
|   `-DeclRefExpr Var 'n'
`-ReturnStmt
  `-IntegerLiteral 'int' 0
```

1.3 Interesting nodes and their hierarchy

1. `ForStmt`. A `for` loop is one node with four fixed child slots, in order: *init* (`DeclStmt`), *condition variable* (here `<<<NULL>>>`, used only for `for (...; T v = ...; )` forms), *condition* (`BinaryOperator <`), *increment* (`UnaryOperator ++`), and finally the *body*. In the dump the body is a `CompoundStmt`, whose only child is the inner `ForStmt`. This is how loop nesting appears in the tree: the inner loop is a child of the outer loop's body, not a sibling.

2. `CallExpr`. `printf("Hello World\n")` is a `CallExpr` whose first child is the callee expression and the remaining children are the arguments. The callee is not a bare `DeclRefExpr`: Clang wraps it in an `ImplicitCastExpr <FunctionToPointerDecay>`, so the node under `CallExpr` is the cast, and the `DeclRefExpr` naming `printf` is its child. The string argument is likewise wrapped in an `ArrayToPointerDecay` cast above the `StringLiteral`. A matcher for "calls to `printf`" therefore has to look through these implicit nodes, which `callee(functionDecl(hasName("printf")))` does for us.

3. `DeclStmt / VarDecl / DeclRefExpr`. `int i = 0;` is a `DeclStmt` (a statement) holding a `VarDecl` (the declaration), whose child is the initializer `IntegerLiteral`. Every later use of `i` is a `DeclRefExpr`, which does not copy the variable but *refers* to a particular `VarDecl` (the dump shows `Var 0x... 'i'` before the addresses were stripped). The program declares `i` twice, so there are two distinct `VarDecl` nodes; each `DeclRefExpr` points to the nearest enclosing one, so the inner loop's `i` shadows the outer one. This declaration/reference link is exactly what is needed to decide whether a local variable is ever used (Part 2, pass 3).

Other nodes worth noting. `ImplicitCastExpr <LValueToRValue>` appears around every variable that is *read* (`i` and `n` in `i < n`) but not around `i` in `i++`, where it is modified in place.

`CompoundStmt` is the { ... } block and just holds a list of statements. The overall root of this dump is `FunctionDecl`, which sits under the `TranslationUnitDecl`.

2 Traversal mechanisms in Clang

`RecursiveASTVisitor`. A CRTP base class. Derive from `RecursiveASTVisitor<Derived>` and define `VisitXxx(Xxx *node)` for the node kinds of interest; the base class walks the whole tree depth-first and calls them. Three layers exist per node kind: `TraverseXxx` (controls descent into children and can prune a subtree by not calling the base), `WalkUpFromXxx` (calls the `Visit` methods for the node and its base classes), and `VisitXxx` (what you usually override; return `false` to stop the whole traversal). Best for imperative analyses that need state across the walk.

AST matchers with `MatchFinder`. A declarative approach: describe the pattern as a composition of matchers (node matchers such as `forStmt`, `varDecl`; narrowing matchers such as `hasName`; traversal matchers such as `hasBody`, `hasCondition`, `hasDescendant`) and attach a `MatchCallback` to it with `MatchFinder::addMatcher`. `.bind("id")` labels sub-nodes, which the callback's `run` retrieves with `Result.Nodes.getNodeAs<T>("id")`. The finder does the walk internally. Concise for local patterns, and what Part 2 uses.

`ASTConsumer` and `ASTFrontendAction`. The hook by which a tool gets the AST at all. A `FrontendAction` creates an `ASTConsumer`, whose `HandleTranslationUnit(ASTContext&)` (or `HandleTopLevelDecl`) is called once parsing finishes. From there one typically starts a `RecursiveASTVisitor` from `Context.getTranslationUnitDecl()`. `newFrontendActionFactory(&Finder)` in a LibTooling `main` hides this for matchers.

`StmtVisitor` / `DeclVisitor`. Dispatch-only visitors: `Visit(node)` calls the `VisitXxx` for that node's dynamic type, but *does not* recurse; you recurse manually via `node->children()`. Used inside Clang itself (e.g. the AST printer, code generation) when the traversal order is custom.

Direct child iteration and parent queries. Every `Stmt` exposes `children()`, and a `DeclContext` exposes `decls()`, for a hand-written walk. Clang nodes have no parent pointer; `ASTContext::getParents(node)` (backed by the `ParentMapContext`) computes the parent map on demand for upward navigation.

3 Part 2: AST-matcher passes

All three matchers are in one file, `pass.cpp`. Each check has its own test file in `tests/`. Every matcher only looks at the main file, so code from headers like `<stdio.h>` is ignored.

3.1 Build and run

The tool is built inside an LLVM 17.0.6 source tree. The target is called `MA4_Pass`, so it does not clash with the lab's `Simple_Pass`.

```
cp -r 2-ast-matcher-passes ~/llvm-project/clang-tools-extra/ma4-passes
# add:  add_subdirectory(ma4-passes)  to clang-tools-extra/CMakeLists.txt
cd ~/llvm-project/build && cmake . && make -j4 MA4_Pass
bin/MA4_Pass ../clang-tools-extra/ma4-passes/tests/assign_in_condition.cpp --
```

`--` means "no compilation database". Tested with LLVM/Clang 17.0.6.

3.2 Pass 1: assignment inside a condition

Goal. Find `if (x = 5)`, where `==` was probably meant.
Approach. We check the condition of an `if`, `for`, `while`, `do-while` and `?:`. In each, we look for an assignment operator `=`. The matcher for `if` is shown below; the other four are the same with a different statement name, and `anyOf` joins all five.

```
ifStmt(hasCondition(ignoringParenImpCasts(
    binaryOperator(hasOperatorName("=")).bind("assignment"))))
```

`ignoringParenImpCasts` skips extra nodes around the condition. In C++, `n = 13` is wrapped in a cast to bool, and `if ((n = 13))` has extra brackets. Without it, these would be missed. We match only plain `=`, not `+=`. The callback prints where the assignment is.
Test. `tests/assign_in_condition.cpp`
Output. Only the assignment on line 5 is reported. The comparison `n == 10` on line 8 is not.

```
tests/assign_in_condition.cpp:5:8
```

3.3 Pass 2: empty loop body

Goal. Find `for`, `while` and `do-while` loops with an empty body {}.
Approach. For each of the three loops, we use `hasBody(compoundStmt(statementCountIs(0)))`. This means the body is a block with no statements inside. Loops with a single statement as the body, or just a `;` (like `for (...);`), are not reported, because the question asks only for empty blocks.
Test. `tests/empty_loop_body.cpp`
Output. Only the empty `for` on line 6 is reported. The `do` and `while` loops in the test have statements in them.

```
tests/empty_loop_body.cpp:6:5
```

3.4 Pass 3: unused local variable

Goal. Find local variables that are declared but never used.
Approach. We use two matchers. One records every local variable when it is declared. The other records every place a local variable is used. Each variable has one declaration node, and every use points back to it, so we can tell variables apart even if they have the same name. We keep two sets: declared variables and used variables. After the whole file is processed, any variable in the declared set but not in the used set is unused, and we print it. We must wait until the end because a variable may be used later in the file than where it is declared. Function parameters, global variables and `static` locals are not reported. A variable that is only assigned to, never read (`int c = 3; c = 4;`), counts as used.
Test. `tests/unused_local.cpp`
Output. The unused `x` in `f` and the unused `x` in `main` are both reported. `n` and `i` are used, so they are not.

```
Unused variable: x
Unused variable: x
```

4 Limitations

- Pass 1 finds only `=`. It misses `if (x += 2)`. It also misses an assignment inside a bigger condition, like `if ((x = 1) && y)`, because we only check the whole condition.
- Pass 2 does not report `for (...);` or `while (...);`, or a body that only holds an empty inner block.
- Pass 3 misses variables that are only assigned and never read. It does not report parameters, globals or `static` locals. It prints only the name, not the line.
- Code in headers is ignored, so problems in included files are not reported. We did not test macros or templates.

Note on AI use

The text of this write-up was drafted with AI assistance (Claude). The passes themselves were written by the authors.